

SPACE INVADERS

Programación Orientada a Objetos (Java)

Sprint: Demo Day + entrega en eGela

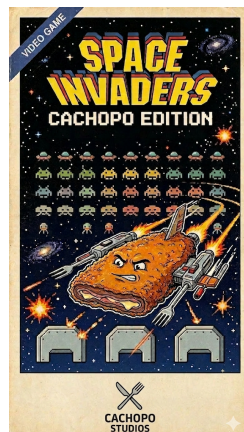
Documentación final

Grupo: Cachopin

11 de mayo de 2026

Estructura según entrega final (Apartados a, b, c, f, g, h).

Los informes de MVC/patrones y de uso de LLM se entregan como documentos aparte.



Índice

1. Introducción: presentación y objetivos	2
1.1. Presentación del proyecto	2
1.2. Objetivos	2
1.2.1. Objetivos de aprendizaje	2
1.2.2. Objetivos de la entrega final	2
2. Herramientas LLM y asistentes de IA	2
2.1. Herramientas utilizadas	2
2.2. Uso en el proyecto	3
2.3. Generación de imágenes y gráficos	3
3. Gestión: reparto de tareas, reuniones y recursos	3
3.1. Metodología y reuniones	3
3.2. Recursos	4
3.3. Reparto general de responsabilidades	4
4. Desarrollo: objetivos y decisiones por sprint	4
4.1. Sprint 1	4
4.2. Sprint 2	5
4.3. Sprint 3	5
5. Reparto de tareas por sprint	5
5.1. Sprint 1	5
5.2. Sprint 2	5
5.3. Sprint 3	6
6. Conclusiones: reflexión y cambios futuros	6
6.1. Reflexión	6
6.2. Cambios y mejoras futuras	6

Introducción: presentación y objetivos

Presentación del proyecto

Space Invaders es un proyecto académico en Java que reproduce el juego clásico: el jugador controla una nave en una cuadrícula y debe eliminar enemigos que descienden, sin ser alcanzado ni dejando que lleguen al borde inferior.

El trabajo se ha organizado en **tres sprints**, con reuniones con el cliente al cierre de cada uno, uso de **Git** para el código y entregas en **eGela**, culminando en el **Demo Day** y esta documentación final.

Objetivos

Objetivos de aprendizaje

- Aplicar **programación orientada a objetos** en un proyecto de tamaño medio.
- Trabajar en **equipo** con reparto de tareas, responsabilidades claras y integración frecuente.
- Practicar **metodología ágil** (planificación por sprints, demo y retroalimentación).
- Entregar un producto **jugable**, estable y alineado con los requisitos hasta la última historia de usuario obligatoria acordada con el cliente.

Objetivos de la entrega final

- Demostrar en el **Demo Day** el estado final del juego y la participación de todo el equipo.
- Aportar la **documentación** requerida en eGela de forma completa y revisada.
- Dejar constancia del **diseño**, la **evolución por sprint** y las **conclusiones** del proyecto.

Herramientas LLM y asistentes de IA

En el desarrollo moderno de software, los **modelos de lenguaje de gran escala (LLM)** son herramientas que pueden mejorar la productividad y la velocidad de desarrollo. Durante *Space Invaders* se utilizaron varias herramientas de este tipo.

Es importante aclarar que la **responsabilidad final del código y del diseño recae en el equipo de desarrollo**: los LLM y los asistentes del IDE fueron **complementarios**, no sustitutos del criterio humano ni de la implementación supervisada.

Herramientas utilizadas

- **Visual Studio Code** con GitHub Copilot
- **Cursor** (IDE con IA integrada)
- **Gemini** (consultas en línea)

- **Claude** (pruebas y preguntas de mejora de código)

Uso en el proyecto

Las herramientas más usadas fueron los **IDE con asistente de IA**. Por un lado, el **autocompletado** ayudó a generar código repetitivo (cabeceras de clases, bucles, esqueletos de métodos, etc.) y sugerencias alineadas con el archivo abierto. Por otro, los **agentes** del IDE (modelos de distintas compañías, p. ej. Anthropic o OpenAI) sirvieron para localizar métodos obsoletos sin borrar, comentar código (revisado después), duplicar o adaptar clases, dar formato a informes $\text{\LaTeX}$ y, puntualmente, añadir `println` de depuración para localizar fallos.

Aun así, **la lógica del juego y las condiciones complejas** requirieron trabajo del equipo; **todo** lo generado o sugerido por estos modelos fue **supervisado** antes de integrarlo.

Generación de imágenes y gráficos

Se ha utilizado la herramienta de generación de imágenes de Gemini (Google) para 2 cuestiones. Esta herramienta se ha utilizado para la generación de las imágenes del fondo del juego en las que el cachopo es una nave.

Se probó también su uso para generar gráficos que explicarán de forma visual y simple la secuencia y la relación entre las diferentes clases al mover al jugador, al hacer tick algún timer y demás (JugadorBueno, Nave, Composite, Espacio...) El resultado no fue muy bueno, por lo que al final estos resúmenes se hicieron en escrito.

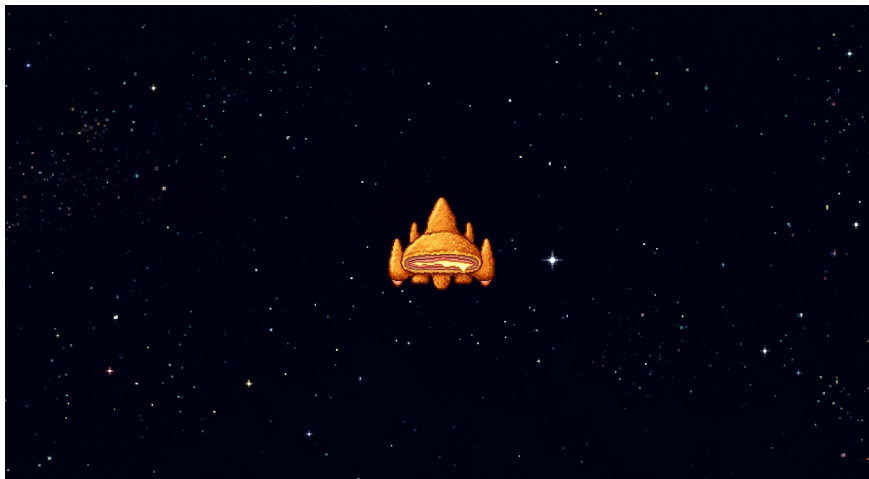


Figura 1: Fondo del juego generado con Gemini: temática *cachopo-nave* en estilo pixel art.

Gestión: reparto de tareas, reuniones y recursos

Metodología y reuniones

Se ha seguido un esquema tipo **SCRUM**: sprints con entrega incremental, lista de tareas priorizada y revisión con el **cliente** (profesorado) al final de cada sprint.

- **Reuniones de sprint:** demostración del incremento (qué funciona), revisión del avance y nota de mejoras para el siguiente ciclo.
- **Comunicación interna:** acuerdos de qué módulos toca cada persona, avisos de bloqueos y integración del código en el repositorio compartido.

Recursos

- **Repositorio:** código versionado en Git (p.ej. GitHub), con historial visible para acreditar la aportación de cada miembro.
- **Entorno:** Java y herramienta de desarrollo acordada por el equipo (IDE, convenciones de estilo, build del proyecto).
- **Documentación interna:** notas de diseño, diagramas y repartos de tareas por sprint (ver también sección 5).

Reparto general de responsabilidades

Ajustar esta lista al reparto real del equipo (nombres y roles).

- **Coordinación / cliente:** preparar demos, asegurar entregas en fecha y calidad mínima del conjunto.
- **Modelo y lógica de juego:** reglas, entidades, colisiones y bucle de juego.
- **Interfaz y control:** pantallas, entrada de teclado, integración con el modelo.
- **Integración y calidad:** pruebas manuales, resolución de conflictos en Git, homogeneidad del código.

Desarrollo: objetivos y decisiones por sprint

Sprint 1

Objetivos principales:

- Tener un **juego jugable mínimo:** nave, enemigo(s), disparo y condiciones de fin de partida.
- Establecer la **arquitectura base** del código y la división en capas de forma coherente.
- Fijar convenciones de paquetes, nombres y flujo de trabajo en Git.

Decisiones destacadas:

- *Completar:* decisiones concretas (p.ej. tamaño de matriz, timers, representación de entidades).

Sprint 2

Objetivos principales:

- **Ampliar** funcionalidad: distintas naves, enemigos y disparos según requisitos del backlog.
- Refactorizar donde haga falta para mantener el código **mantenible** sin romper lo ya entregado.

Decisiones destacadas:

- *Completar*: cómo se encapsuló la variación de comportamiento (estrategias, fábricas, composición de figuras, etc.) sin duplicar documentación del informe técnico adjunto.

Sprint 3

Objetivos principales:

- Cerrar **historias de usuario** pendientes, pulir la experiencia de juego y la estabilidad.
- Completar **documentación final**, preparar **Demo Day** y entrega en eGela.
- Opcional: extensiones acordadas para la nota máxima (HU adicional), si aplica.

Decisiones destacadas:

- *Completar*: últimas mejoras, priorización entre “pulido” y “nueva funcionalidad”.

Reparto de tareas por sprint

En paralelo a la gestión global (sección 3), el equipo documenta el reparto mediante tablas como la siguiente (**Sprint**, **tarea**, **responsables**, tiempo planificado *vs.* real y **comentarios**). Añadir una fila por cada tarea. Los PDFs en DOCUMENTOS/REPARTO TAREAS/ pueden ampliar el detalle.

Sprint 1

* Reparto de tareas

Sprint	Tarea	Responsables	Tiempo planificado	Tiempo real	Comentarios
1	Mover la nave	Ander	1h	4h	Parecía fácil...

Sprint 2

*** Reparto de tareas**

Sprint	Tarea	Responsables	Tiempo planificado	Tiempo real	Comentarios
2	<i>(sustituir por tareas reales)</i>				
2					

Sprint 3*** Reparto de tareas**

Sprint	Tarea	Responsables	Tiempo planificado	Tiempo real	Comentarios
3	<i>(sustituir por tareas reales)</i>				

Conclusiones: reflexión y cambios futuros**Reflexión**

Completar con la reflexión del grupo: qué se ha aprendido trabajando con POO en equipo, qué ha costado más (integración, diseño, plazos), y cómo se ha mejorado la coordinación a lo largo de los sprints.

Cambios y mejoras futuras

Ideas razonables para continuidad del proyecto (sin obligación de implementarlas):

- Pruebas automatizadas (p. ej. JUnit) sobre lógica del modelo.
- Puntuación, niveles o dificultad configurable.
- Mejoras de interfaz o contenido audiovisual.
- Optimización de colisiones o de refresco de la vista si el crecimiento del código lo hace necesario.

Fin del documento (entrega documentación final + Demo Day).